Rúnar: A Multi-Frontend Compiler from High-Level Smart Contract Languages to Bitcoin Script

A Formally Structured Compilation Pipeline with
Cross-Compiler Conformance Guarantees

Siggi Oskarsson, BSV Association

March 2026

Source code: `https://github.com/icellan/runar`

Abstract

We present Rúnar, a compiler that translates smart contracts written in strict subsets of five high-level languages—TypeScript, Solidity, Move, Go, and Rust—into Bitcoin SV Script. The compilation proceeds through a six-phase pipeline: parsing, validation, type checking, A-Normal Form (ANF) lowering, stack lowering, and opcode emission. All five frontends converge onto a shared Abstract Syntax Tree, which is then lowered through an ANF-based intermediate representation that serves as the *conformance boundary* between seven independent compiler implementations. We formalize the type system, describe the stack scheduling algorithm that maps named temporaries to positional stack operations, and characterize the correctness properties guaranteed by the pipeline: type safety, termination, determinism, and UTXO safety. The multi-frontend architecture demonstrates that language-agnostic smart contract compilation to a stack-based target is both feasible and practical, enabling developers to write provably equivalent Bitcoin scripts regardless of source language.

Contents

1 Introduction **3**

2 Source Language **3**
- 2.1 Grammar . . . 3
- 2.2 Type System . . . 4
  - 2.2.1 Typing Rules . . . 5
  - 2.2.2 Built-in Function Typing . . . 5
- 2.3 Contract Model . . . 6

3 Multi-Frontend Architecture **7**
- 3.1 TypeScript Frontend . . . 7
- 3.2 Solidity-like Frontend . . . 7
- 3.3 Move-style Frontend . . . 8
- 3.4 Go Frontend . . . 9
- 3.5 Rust Frontend . . . 9
- 3.6 Frontend Convergence . . . 9

4 Compilation Pipeline **10**
4.1 Pass 1: Parse . . . 10
4.2 Pass 2: Validate . . . 10
4.3 Pass 3: Type Check . . . 11
4.4 Pass 4: ANF Lowering . . . 11
4.4.1 Logical Operator Lowering . . . 11
4.4.2 Stateful Contract Injection . . . 12
4.5 Pass 4.5: Constant Folding (Optimizer) . . . 12
4.6 Pass 5: Stack Lowering . . . 13
4.6.1 Stack Model . . . 13
4.6.2 Last-Use Analysis . . . 13
4.6.3 Value Retrieval . . . 13
4.6.4 Lowering Bindings . . . 13
4.7 Pass 6: Emit . . . 13

5 Intermediate Representations **14**
5.1 Contract AST . . . 14
5.2 ANF IR . . . 14
5.3 Stack IR . . . 15

6 Bitcoin Script Code Generation **15**
6.1 Operator Compilation . . . 15
6.2 Built-in Function Compilation . . . 16
6.2.1 Single-Opcode Built-ins . . . 16
6.2.2 Multi-Opcode Built-ins . . . 16
6.3 Method Dispatch . . . 17
6.4 Terminal Assert Pattern . . . 17
6.5 Loop Compilation . . . 17

7 Stateful Contract Compilation **18**
7.1 The OP_PUSH_TX Pattern . . . 18
7.2 State Continuation . . . 18
7.3 Multi-Output Contracts . . . 18

8 Correctness Properties **18**

9 Post-Quantum Signature Verification (Experimental) **19**
9.1 WOTS+ (Winternitz One-Time Signature) . . . 19
9.2 SLH-DSA (FIPS 205, SPHINCS+ SHA-256) . . . 20

10 Empirical Evaluation **20**
10.1 Test Corpus . . . 21
10.2 Cross-Compiler Conformance Results . . . 21
10.3 Script Size Measurements . . . 21
10.4 Test Infrastructure . . . 22
10.5 Deployment SDKs . . . 22

11 Limitations and Threats to Validity **23**

12 Related Work **24**

13 Conclusion **24**

1 Introduction

Bitcoin Script is a stack-based, non-Turing-complete language embedded in every Bitcoin transaction. It governs the conditions under which unspent transaction outputs (UTXOs) may be spent, forming the basis of Bitcoin's programmable money. Despite its expressive power—particularly in Bitcoin SV, where the original opcodes have been restored—Bitcoin Script remains difficult to program directly. Stack manipulation is error-prone, the absence of named variables forces manual bookkeeping, and the low-level instruction set lacks the abstractions that developers expect from modern languages.

RÚNAR addresses these challenges through a multi-pass compiler that accepts smart contracts written in high-level languages and produces Bitcoin Script locking scripts. The key design decisions are:

(i) **Multi-frontend convergence.** Five distinct source syntaxes (TypeScript, Solidity-like, Move-style, Go, Rust) are parsed into a single shared AST, enabling polyglot development with identical compilation output.

(ii) **ANF as conformance boundary.** The A-Normal Form intermediate representation is fully specified and canonically serializable, enabling seven independent compiler implementations (TypeScript, Go, Rust, Python, Zig, Ruby, Java) to produce byte-identical output for the same input.

(iii) **Static guarantees.** The type system, combined with validation constraints (bounded loops, no recursion, affine consumption tracking for signature and preimage types), ensures that all compiled programs terminate, maintain type safety, and preserve UTXO integrity.

(iv) **Deterministic stack scheduling.** A last-use analysis algorithm deterministically maps named ANF temporaries to Bitcoin Script stack positions, producing identical bytecode across all seven compiler implementations.

The remainder of this paper is organized as follows. Section 2 defines the source language, its type system, and the contract model. Section 3 describes each of the five frontend parsers and their syntax-to-AST mappings. Section 4 formalizes the six-phase compilation pipeline. Section 5 specifies the three intermediate representations (AST, ANF IR, Stack IR). Section 6 details Bitcoin Script code generation, including the stack scheduling algorithm and opcode emission. Section 7 describes the compilation of stateful contracts using the OP_PUSH_TX pattern. Section 8 states the correctness properties of the compilation. Section 10 presents empirical evaluation results including cross-compiler conformance and script size measurements. Section 11 discusses limitations and threats to validity. Section 12 surveys related work, and Section 13 concludes.

2 Source Language

The RÚNAR source language is a strict, decidable subset of TypeScript augmented with domain-specific types for Bitcoin Script operations. This section formalizes the grammar, type system, and contract model.

2.1 Grammar

A RÚNAR program consists of a single contract class extending one of two base classes. The grammar is given in Figure 1.

The grammar enforces several structural restrictions that are verified statically during compilation: exactly one contract per file, a constructor that calls `super()` as its first statement,

⟨*program*⟩ ::= ⟨*contract*⟩

⟨*contract*⟩ ::= '`class`' ⟨*id*⟩ '`extends`' ⟨*base-class*⟩ '`{`' ⟨*member*⟩* '`}`'

⟨*base-class*⟩ ::= '`SmartContract`' | '`StatefulSmartContract`'

⟨*member*⟩ ::= ⟨*property*⟩ | ⟨*constructor*⟩ | ⟨*method*⟩

⟨*property*⟩ ::= ['`readonly`'] ⟨*id*⟩ '`:`' ⟨*type*⟩

⟨*constructor*⟩ ::= '`constructor`' '`(`' ⟨*params*⟩ '`)`' '`{`' '`super(`' ⟨*args*⟩ '`)`' '`;`' ⟨*assign*⟩* '`}`'

⟨*method*⟩ ::= ['`public`'] ⟨*id*⟩ '`(`' ⟨*params*⟩ '`)`' ['`:`' ⟨*type*⟩] '`{`' ⟨*stmt*⟩* '`}`'

⟨*stmt*⟩ ::= ⟨*var-decl*⟩ | ⟨*assign-stmt*⟩ | ⟨*if-stmt*⟩ | ⟨*for-stmt*⟩
| ⟨*return-stmt*⟩ | ⟨*expr-stmt*⟩

⟨*var-decl*⟩ ::= ('`const`' | '`let`') ⟨*id*⟩ ['`:`' ⟨*type*⟩] '`=`' ⟨*expr*⟩

⟨*if-stmt*⟩ ::= '`if`' '`(`' ⟨*expr*⟩ '`)`' '`{`' ⟨*stmt*⟩* '`}`' ['`else`' '`{`' ⟨*stmt*⟩* '`}`']

⟨*for-stmt*⟩ ::= '`for`' '`(`' ⟨*var-decl*⟩ '`;`' ⟨*expr*⟩ '`;`' ⟨*expr*⟩ '`)`' '`{`' ⟨*stmt*⟩* '`}`'

⟨*expr*⟩ ::= ⟨*literal*⟩ | ⟨*id*⟩ | ⟨*unary-expr*⟩ | ⟨*binary-expr*⟩ | ⟨*ternary-expr*⟩
| ⟨*call-expr*⟩ | ⟨*member-expr*⟩ | ⟨*index-expr*⟩ | '`(`' ⟨*expr*⟩ '`)`'

⟨*binary-op*⟩ ::= '`+`' | '`-`' | '`*`' | '`/`' | '`%`' | '`===`' | '`!==`' | '`<`' | '`>`'
| '`<=`' | '`>=`' | '`&&`' | '`||`' | '`&`' | '`|`' | '`^`' | '`<<`' | '`>>`'

⟨*unary-op*⟩ ::= '`!`' | '`-`' | '`~`'

⟨*type*⟩ ::= '`bigint`' | '`boolean`' | '`ByteString`' | '`PubKey`' | '`Sig`'
| '`Sha256`' | '`Ripemd160`' | '`Addr`' | '`SigHashPreimage`'
| '`RabinSig`' | '`RabinPubKey`' | '`FixedArray<`' ⟨*type*⟩ '`,`' ⟨*nat*⟩ '`>`'
| '`void`'

Figure 1: Abstract grammar of the RÚNAR source language.

and a restricted set of control flow constructs (no `while`, `switch`, `try/catch`, or unbounded iteration).

2.2 Type System

The RÚNAR type system is a simple, decidable type system with subtyping and targeted affine constraints on security-critical types. Types are organized into two families with a strict separation between them.

Definition 2.1 (Type Universe)**.** *The set of RÚNAR types $\mathcal{T}$ is partitioned into:*

$$\mathcal{T}_{\text{int}} = \{\texttt{bigint}, \texttt{RabinSig}, \texttt{RabinPubKey}\} \tag{1}$$

$$\mathcal{T}_{\text{bytes}} = \{\texttt{ByteString}, \texttt{PubKey}, \texttt{Sig}, \texttt{Sha256}, \texttt{Ripemd160}, \texttt{Addr}, \texttt{SigHashPreimage}\} \tag{2}$$

$$\mathcal{T}_{\text{bool}} = \{\texttt{boolean}\} \tag{3}$$

$$\mathcal{T}_{\text{array}} = \{\texttt{FixedArray}\langle T, n\rangle \mid T \in \mathcal{T}, n \in \mathbb{N}\} \tag{4}$$

$$\mathcal{T}_{\text{void}} = \{\texttt{void}\} \tag{5}$$

Definition 2.2 (Subtyping Relation)**.** *The subtyping relation* $<:$ *is the reflexive-transitive closure of:*

$$\mathit{RabinSig} <: \mathit{bigint} \tag{6}$$
$$\mathit{RabinPubKey} <: \mathit{bigint} \tag{7}$$
$$\mathit{PubKey} <: \mathit{ByteString} \tag{8}$$
$$\mathit{Sig} <: \mathit{ByteString} \tag{9}$$
$$\mathit{Sha256} <: \mathit{ByteString} \tag{10}$$
$$\mathit{Ripemd160} <: \mathit{ByteString} \tag{11}$$
$$\mathit{Addr} <: \mathit{ByteString} \tag{12}$$
$$\mathit{SigHashPreimage} <: \mathit{ByteString} \tag{13}$$

This subtyping lattice is shallow (depth at most 1) and decidable in constant time. Cross-family comparisons are rejected: a value of type `bigint` cannot be compared with a value of type `ByteString` using relational operators.

Definition 2.3 (Affine Types)**.** *Types* `SigHashPreimage` *and* `Sig` *are* affine*: values of these types must be consumed at most once. The type checker tracks usage and rejects programs that duplicate these values.*

The affine constraint on `SigHashPreimage` prevents the sighash preimage from being duplicated, which would enable transaction malleability attacks. In practice, the preimage is consumed exactly once because the compiler auto-injects `checkPreimage` for stateful contracts, but the type-level guarantee is the weaker affine bound (at most once). The affine constraint on `Sig` similarly prevents signature duplication attacks.

2.2.1 Typing Rules

We present selected typing rules using a standard judgment form $\Gamma \vdash e : \tau$, where Γ is a typing environment mapping identifiers to types.

$$\frac{n \in \mathbb{Z}}{\Gamma \vdash n\mathtt{n} : \mathtt{bigint}} \quad [\text{T-BigInt}] \qquad \frac{b \in \{\mathtt{true}, \mathtt{false}\}}{\Gamma \vdash b : \mathtt{boolean}} \quad [\text{T-Bool}] \tag{14}$$

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \quad [\text{T-Var}] \qquad \frac{\Gamma \vdash e : \tau' \quad \tau' <: \tau}{\Gamma \vdash e : \tau} \quad [\text{T-Sub}] \tag{15}$$

$$\frac{\Gamma \vdash e_1 : \mathtt{bigint} \quad \Gamma \vdash e_2 : \mathtt{bigint} \quad \oplus \in \{+, -, *, /, \%, \&, |, \hat{}, \ll, \gg\}}{\Gamma \vdash e_1 \oplus e_2 : \mathtt{bigint}} \quad [\text{T-Arith}] \tag{16}$$

$$\frac{\Gamma \vdash e_1 : \mathtt{bigint} \quad \Gamma \vdash e_2 : \mathtt{bigint} \quad \oplus \in \{<, >, \leq, \geq\}}{\Gamma \vdash e_1 \oplus e_2 : \mathtt{boolean}} \quad [\text{T-Rel}] \tag{17}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2 \quad \tau_1 \sim \tau_2 \quad \oplus \in \{===, !==\}}{\Gamma \vdash e_1 \oplus e_2 : \mathtt{boolean}} \quad [\text{T-Eq}] \tag{18}$$

where $\tau_1 \sim \tau_2$ (type compatibility) holds when $\tau_1 <: \tau_2$ or $\tau_2 <: \tau_1$, and both types belong to the same family ($\mathcal{T}_{\text{int}}$ or $\mathcal{T}_{\text{bytes}}$ or $\mathcal{T}_{\text{bool}}$).

2.2.2 Built-in Function Typing

Rúnar provides a closed set of built-in functions. No user-defined free-standing functions are permitted; the type checker rejects calls to any function not in this set or not a method on the current contract.

Category	Function	Signature
Hashing	`sha256`	`ByteString` → `Sha256`
	`hash160`	`ByteString` → `Ripemd160`
	`hash256`	`ByteString` → `Sha256`
	`ripemd160`	`ByteString` → `Ripemd160`
Signatures	`checkSig`	`Sig` × `PubKey` → `boolean`
	`checkMultiSig`	`Sig[]` × `PubKey[]` → `boolean`
	`verifyRabinSig`	`ByteString` × `RabinSig` × `ByteString` × `RabinPubKey` → `boolean`
Mathematics	`abs`, `min`, `max`	$\texttt{bigint}^{(1\text{–}2)} \to \texttt{bigint}$
	`within`	`bigint` × `bigint` × `bigint` → `boolean`
	`pow`, `sqrt`, `gcd`	$\texttt{bigint}^{(1\text{–}2)} \to \texttt{bigint}$
	`safediv`, `safemod`, `mulDiv`	$\texttt{bigint}^{(2\text{–}3)} \to \texttt{bigint}$
Byte Ops	`len`	`ByteString` → `bigint`
	`cat`	`ByteString` × `ByteString` → `ByteString`
	`num2bin`, `bin2num`	`bigint` × `bigint` → `ByteString` / `ByteString` → `bigint`
Post-Quantum	`verifyWOTS`	`ByteString` × `ByteString` × `ByteString` → `boolean`
Control	`assert`	`boolean` → `void`

Table 1: Selected built-in function signatures (abbreviated).

2.3 Contract Model

RÚNAR defines two contract base classes that correspond to distinct Bitcoin Script patterns.

Definition 2.4 (Stateless Contract)**.** *A* stateless contract *(`SmartContract`) has the following properties:*

(a) All properties are **`readonly`** *and embedded as constants in the locking script at deployment time.*

(b) Public methods are spending conditions: each is a predicate that must evaluate to **`true`** *for the UTXO to be spent.*

(c) No state transitions occur; the script is evaluated once and discarded.

Definition 2.5 (Stateful Contract)**.** *A* stateful contract *(`StatefulSmartContract`) extends the stateless model with:*

(a) At least one mutable (non-`readonly`*) property.*

(b) Automatic injection of `checkPreimage` *verification at the entry of every public method, implementing the OP_PUSH_TX pattern.*

(c) Automatic state continuation: at method exit, the compiler emits code that serializes the updated state, constructs the expected transaction output, and verifies it matches the spending transaction's output hash.

The `parentClass` field on the AST's `ContractNode` discriminates between these two models, determining which compilation strategy is applied in later passes.

3 Multi-Frontend Architecture

A key design contribution of RÚNAR is the separation of surface syntax from the compilation pipeline. Five distinct frontend parsers translate source code into a shared AST representation, after which the remaining five passes are syntax-agnostic. This section describes each frontend's syntax and its mapping to the canonical AST.

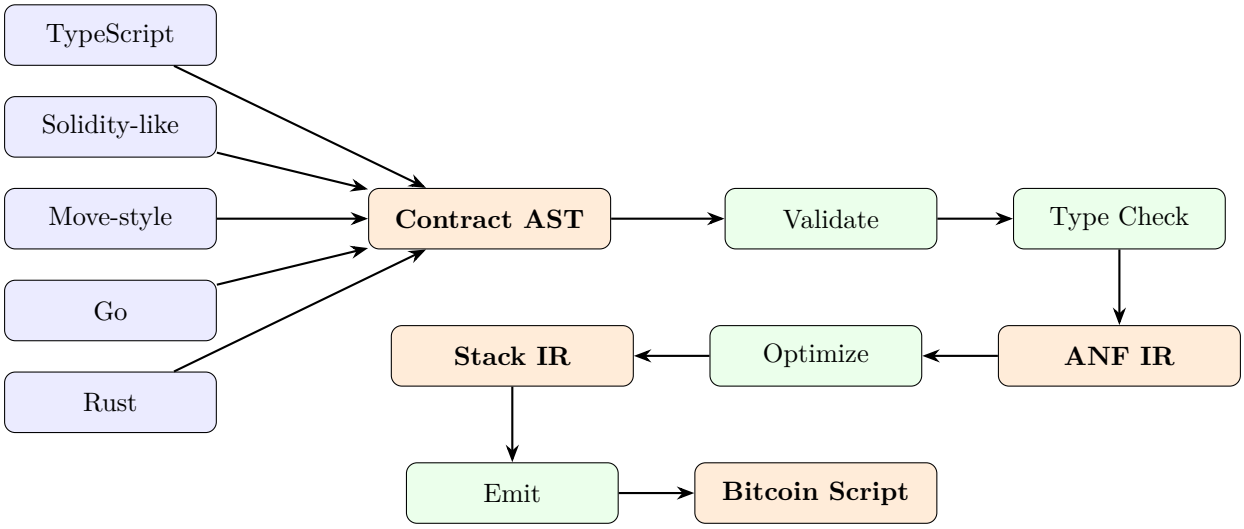


Figure 2: Multi-frontend compilation pipeline. All five frontends converge onto the shared Contract AST; subsequent passes are syntax-agnostic.

3.1 TypeScript Frontend

The canonical RÚNAR syntax uses a strict subset of TypeScript. Contracts are declared as classes extending one of the two base classes:

```
class P2PKH extends SmartContract {
  readonly pubKeyHash: Ripemd160;

  constructor(pubKeyHash: Ripemd160) {
    super(pubKeyHash);
    this.pubKeyHash = pubKeyHash;
  }

  public unlock(sig: Sig, pubKey: PubKey) {
    assert(hash160(pubKey) === this.pubKeyHash);
    assert(checkSig(sig, pubKey));
  }
}
```

The TypeScript frontend maps directly to the AST with minimal transformation: `readonly` maps to the property's `readonly` flag, `public`/`private` keywords control method visibility, and TypeScript type annotations are preserved verbatim as RÚNAR types. Numeric literals (both `42n` and `42`) are uniformly interpreted as `bigint`.

3.2 Solidity-like Frontend

The Solidity-like frontend accepts a syntax inspired by Ethereum's Solidity, with adaptations for the RÚNAR type system:

```
pragma runar ^0.1.0;

contract P2PKH is SmartContract {
  Ripemd160 immutable pubKeyHash;

  function unlock(Sig sig, PubKey pubKey) public {
    require(hash160(pubKey) == pubKeyHash);
    require(checkSig(sig, pubKey));
  }
}
```

Table 2 summarizes the key syntactic mappings.

Solidity Syntax	AST Mapping	Notes
`immutable`	`readonly: true`	Before field name
`require(e)`	`assert(e)`	Direct substitution
`==`, `!=`	`===`, `!==`	Strict equality
`int`, `uint`, `int256`	`bigint`	All integer types unified
`bool`	`boolean`	Direct mapping
`bytes`	`ByteString`	Direct mapping
`address`	`Addr`	20-byte type
`Type[N]`	`FixedArray<Type,N>`	Array syntax

Table 2: Solidity-like frontend: syntax-to-AST mappings.

The parser is implemented as a hand-written recursive descent parser with a custom lexer. A precedence-climbing algorithm handles operator precedence. The constructor is auto-generated from the struct fields if not explicitly provided, with `super()` injected as the first statement.

3.3 Move-style Frontend

The Move-style frontend uses a syntax inspired by the Move language, with module declarations and resource structs:

```
module P2PKH {
  use runar::SmartContract;

  resource struct P2PKH {
    pub_key_hash: Ripemd160 readonly,
  }

  public fun unlock(sig: Sig, pub_key: PubKey) {
    assert!(hash160(pub_key) == self.pub_key_hash);
    assert!(check_sig(sig, pub_key));
  }
}
```

A distinguishing feature of the Move frontend is automatic identifier conversion from `snake_case` to `camelCase`: `pub_key_hash` becomes `pubKeyHash`, `check_sig` becomes `checkSig`, and so forth. The `assert!` macro is mapped to `assert()`, and `assert_eq!(a, b)` is desugared to `assert(a === b)`.

Type mappings include `Int` → `bigint`, `Bool` → `boolean`, and `vector` → `ByteString`. Reference types (`&T`, `&mut T`) are stripped during parsing. The presence of mutable fields (`&mut`) or the use of `StatefulSmartContract` determines the contract model.

3.4 Go Frontend

The Go frontend parses Go source code using the standard Go parser from the `go/parser` package, then maps the Go AST to the RÚNAR AST:

```
type P2PKH struct {
  runar.SmartContract
  PubKeyHash runar.Ripemd160Hash `runar:"readonly"`
}

func (c *P2PKH) Unlock(sig runar.Sig, pubKey runar.PubKey) {
  runar.Assert(runar.Hash160(pubKey) == c.PubKeyHash)
  runar.Assert(runar.CheckSig(sig, pubKey))
}
```

Key mapping rules include:

- **Embedding**: `runar.SmartContract` as an anonymous field determines the base class.
- **Struct tags**: `` `runar:"readonly"` `` marks immutable properties.
- **Visibility**: Exported (capitalized) methods are public; unexported methods are private.
- **Name conversion**: `PascalCase` → `camelCase` for method and field names.
- **Selector resolution**: `runar.X` selectors are mapped to built-in function calls; non-`runar` selectors are rejected.
- **Type conversions**: `runar.Int(0)` unwraps to the inner value; `runar.Bool(expr)` similarly unwraps.

3.5 Rust Frontend

The Rust frontend uses attribute macros to annotate standard Rust structs and impl blocks:

```
#[runar::contract]
pub struct P2PKH {
  #[readonly]
  pub pub_key_hash: Ripemd160,
}

impl P2PKH {
  pub fn unlock(&self, sig: &Sig, pub_key: &PubKey) {
    assert!(hash160(pub_key) == self.pub_key_hash);
    assert!(check_sig(sig, pub_key));
  }
}
```

The Rust frontend shares the `snake_case` → `camelCase` conversion with the Move frontend. Reference types (`&T`) are stripped, and `.clone()` calls are treated as no-ops (necessary for Rust's borrow checker but irrelevant to Bitcoin Script semantics). The `#[readonly]` attribute on fields is stripped by the proc-macro (Rust does not allow `proc_macro_attribute` on struct fields natively). Methods live in a plain `impl` block, and `pub fn` versus `fn` controls method visibility. Range-based for loops (`for i in 0..n`) are translated to bounded C-style loops in the AST.

3.6 Frontend Convergence

Property 3.1 (Frontend Equivalence)**.** *For any contract C expressible in all five frontend syntaxes, let $\mathit{parse}_f(C_f)$ denote the AST produced by frontend f. Then:*

$$\forall f_1, f_2 \in \{\mathit{TS}, \mathit{Sol}, \mathit{Move}, \mathit{Go}, \mathit{Rust}\} : \quad \mathit{normalize}(\mathit{parse}_{f_1}(C_{f_1})) = \mathit{normalize}(\mathit{parse}_{f_2}(C_{f_2}))$$

where normalize canonicalizes identifier names (via the `camelCase` convention), strips source locations, and orders properties by declaration.

This property is verified by the cross-compiler conformance test suite, which compiles the same contracts in all available formats and compares the byte-identical output at the ANF IR and Bitcoin Script levels.

4 Compilation Pipeline

The RÚNAR compilation pipeline consists of six pure-function passes. Each pass transforms a well-defined input into a well-defined output, with no shared mutable state between passes.

4.1 Pass 1: Parse

The parser accepts a source string and an optional filename. The filename's extension determines which frontend parser is invoked:

Extension	Parser	Available in
`.runar.ts` (or default)	TypeScript (via ts-morph)	TS, Go, Rust
`.runar.sol`	Solidity-like (recursive descent)	TS, Go, Rust
`.runar.move`	Move-style (recursive descent)	TS, Go, Rust
`.runar.go`	Go (`go/parser` + AST mapping)	Go only
`.runar.rs`	Rust (tokenizer + recursive descent)	Rust only

The TypeScript reference compiler dispatches `.runar.sol` and `.runar.move` to their respective hand-written parsers, with all other extensions falling through to the TypeScript parser. The `.runar.go` and `.runar.rs` formats are handled exclusively by the Go and Rust compilers respectively, which leverage their native language toolchains for parsing.

All parsers produce a `ContractNode` AST (defined in Section 5.1) or a list of diagnostic errors. The parser extracts exactly one contract per source file.

4.2 Pass 2: Validate

The validation pass enforces language subset constraints without modifying the AST. The following invariants are checked:

V1. All property types are valid RÚNAR types (no `void`, `any`, `number`, `string`).

V2. The constructor calls `super()` as its first statement.

V3. Each property is assigned exactly once in the constructor.

V4. All properties in `SmartContract` are `readonly`. If a `StatefulSmartContract` has no mutable properties, a warning is emitted (suggesting `SmartContract` instead), but compilation proceeds.

V5. No direct or mutual recursion among methods.

V6. All `for` loop bounds are compile-time constants.

V7. No disallowed constructs: `while`, `do-while`, `switch`, `try/catch`, `async/await`, decorators, closures, `typeof`, `instanceof`, or arbitrary imports.

Proposition 4.1 (Termination from Validation)**.** *If a program passes validation, its execution terminates. This follows from V6 (all loops are bounded) and V5 (no recursion), which together ensure that the call graph is acyclic and all iteration is finite.*

4.3 Pass 3: Type Check

The type checker verifies type consistency according to the rules in Section 2.2. It proceeds in four phases:

1. **Declaration collection**: Build a typing environment Γ from contract properties.
2. **Constructor verification**: Check `super()` arguments and property assignments.
3. **Method body checking**: For each method, extend Γ with parameters and local variables; type-check each statement and expression.
4. **Whole-program checks**: Verify affine type consumption, code size bounds, and unknown function rejection.

The type checker maintains a map of approximately 50 built-in function signatures. Any call to a function not in this map, not a contract method, and not a local variable is rejected. This *closed-world assumption* ensures that the compiled script contains no references to functions that cannot be realized in Bitcoin Script.

4.4 Pass 4: ANF Lowering

The ANF lowering pass transforms the typed AST into A-Normal Form, a representation in which every sub-expression is bound to a named temporary:

Definition 4.2 (A-Normal Form)**.** *An ANF program consists of a sequence of* bindings $b_0, b_1, \ldots, b_n$ *where each binding has the form:*

$$\texttt{let}\ t_i = V_i$$

and V_i is a simple value*—an operation whose arguments are exclusively variable references (to parameters, properties, or earlier temporaries t_j where $j < i$).*

The transformation eliminates all nested expressions. For example:

```
assert(hash160(pubKey) === this.pubKeyHash)
```

becomes:

$$\begin{aligned}
&\texttt{let}\ t_0 = \texttt{load_param(pubKey)}\\
&\texttt{let}\ t_1 = \texttt{call(hash160, [}t_0\texttt{])}\\
&\texttt{let}\ t_2 = \texttt{load_prop(pubKeyHash)}\\
&\texttt{let}\ t_3 = \texttt{bin_op(===, }t_1, t_2\texttt{)}\\
&\texttt{let}\ t_4 = \texttt{assert(}t_3\texttt{)}
\end{aligned}$$

Temporaries are numbered sequentially ($t_0, t_1, t_2, \ldots$) within each method. This canonical naming scheme is critical for cross-compiler conformance: all seven compiler implementations must produce identical temporary indices for the same input.

4.4.1 Logical Operator Lowering

Unlike many compilers, RÚNAR does *not* desugar logical operators into short-circuit conditional expressions. Instead, `&&` and `||` are preserved as binary operations through the ANF IR and lowered directly to `OP_BOOLAND` and `OP_BOOLOR` respectively. Both operands are always evaluated. This is safe because RÚNAR programs are side-effect-free: the language permits no I/O, no mutation outside of explicit property updates, and no non-terminating expressions, so eager evaluation of both operands is semantically equivalent to short-circuit evaluation.

4.4.2 Stateful Contract Injection

For contracts extending `StatefulSmartContract`, the ANF lowering pass automatically injects additional bindings at the entry and exit of every public method:

Entry injection:

$$\begin{aligned}
&\texttt{let } t_0 = \texttt{load_param(txPreimage)} \\
&\texttt{let } t_1 = \texttt{check_preimage}(t_0) \\
&\texttt{let } t_2 = \texttt{assert}(t_1)
\end{aligned}$$

Exit injection (single-output state continuation):

$$\begin{aligned}
\texttt{let } t_k &= \texttt{get_state_script()} \\
\texttt{let } t_{k+1} &= \texttt{call(hash256, [}t_k\texttt{])} \\
\texttt{let } t_{k+2} &= \texttt{load_param(txPreimage)} \\
\texttt{let } t_{k+3} &= \texttt{call(extractOutputHash, [}t_{k+2}\texttt{])} \\
\texttt{let } t_{k+4} &= \texttt{bin_op(===, } t_{k+1}, t_{k+3}\texttt{)} \\
\texttt{let } t_{k+5} &= \texttt{assert}(t_{k+4})
\end{aligned}$$

4.5 Pass 4.5: Constant Folding (Optimizer)

Between ANF lowering and stack lowering, a constant folding optimizer is available to evaluate compile-time-known expressions:

Definition 4.3 (Constant Folding)**.** *A binding* `let` $t_i = V_i$ *is* foldable *if V_i is a binary operation, unary operation, or pure built-in call whose arguments are all constants (i.e.,* `load_const` *bindings). The optimizer replaces V_i with a* `load_const` *of the computed result.*

The optimizer handles all arithmetic, comparison, and bitwise operations on constants, as well as pure mathematical built-ins (`abs`, `min`, `max`, `pow`, `sqrt`, `gcd`, etc.). Constants are propagated through the binding sequence, enabling cascading optimizations.

The optimizer preserves the binding indices—it replaces values in-place rather than removing bindings—ensuring that subsequent bindings' references remain valid.

Conformance tradeoff: on by default; checked-in goldens are fold-OFF. The constant folding optimizer is *enabled by default* in the production compilation pipeline, across all seven independent compiler implementations (Section 10.2). Each compiler's CLI additionally exposes an explicit `-disable-constant-folding` opt-out, because the checked-in conformance goldens—the fixed ANF IR and Bitcoin Script hex files used to verify byte-identical cross-compiler output against Theorem 8.4—were stamped with folding disabled, to pin a stable byte-exact baseline for golden-file comparison. Folding is reproduced identically by all seven compilers, so enabling it does not itself break the byte-identical cross-compiler guarantee; it only diverges from the pre-recorded fold-OFF goldens, which is why the goldens and the production default are allowed to disagree.

In practice, the conformance suite exercises both modes: the baseline multi-format run passes `-disable-constant-folding` and checks cross-tier hex and ANF parity against the goldens, while a companion run re-executes the same fixtures with folding on and separately enforces cross-tier parity across all seven tiers, without comparing against the fold-OFF goldens. When enabled, the optimizer correctly folds all arithmetic ($+$, $-$, $*$, $/$, $\%$), comparison ($<$, $>$, $\leq$, $\geq$, $===$, $!==$), bitwise (&, |, $\wedge$, $\ll$, $\gg$), and unary ($-$, !, $\sim$) operations, as well as pure built-in function calls (`abs`, `min`, `max`, `pow`, `sqrt`, `gcd`, `sign`, `clamp`, `divmod`, `log2`, among others). Disabling folding instead passes the unmodified ANF IR directly to stack lowering, preserving the canonical form that the checked-in goldens were stamped against.

4.6 Pass 5: Stack Lowering

The stack lowering pass is the most complex transformation in the pipeline. It converts the named-variable ANF representation into a positional stack representation suitable for Bitcoin Script emission.

4.6.1 Stack Model

Bitcoin Script provides a main stack and an alt stack. The compiler models the main stack as a sequence of named values indexed from the top (position 0) downward:

Definition 4.4 (Stack Map)**.** *A* stack map $\sigma : \mathbb{N} \to$ *Name maps each stack position (0 = top) to the name of the value stored there. The compiler maintains σ as a virtual tracking structure during lowering.*

4.6.2 Last-Use Analysis

Before generating stack operations, the compiler performs a last-use analysis:

Definition 4.5 (Last Use)**.** *For each variable v defined at binding index d and referenced at binding indices $u_1, u_2, \ldots, u_k$ (where $u_1 < u_2 < \cdots < u_k$), the* last use *of v is u_k. At binding u_k, the value of v may be consumed (removed from the stack); at all earlier uses, it must be copied.*

This analysis determines whether a value is accessed via `OP_ROLL` (consuming, for last use) or `OP_PICK` (copying, for non-last use). The choice between these operations is the primary source of optimization in stack lowering.

4.6.3 Value Retrieval

To bring a value to the top of the stack, the compiler uses Algorithm 1.

4.6.4 Lowering Bindings

Each ANF binding is lowered by:

1. Bringing required operands to the top of the stack using Algorithm 1, consulting the last-use analysis to determine whether to consume or copy.
2. Emitting the appropriate opcode(s) for the operation.
3. Recording the result's name in the stack map.

4.7 Pass 6: Emit

The emission pass serializes Stack IR operations into Bitcoin Script bytecode. Two peephole optimizations are applied during emission:

1. **Verify fusion**: `OP_EQUAL` `OP_VERIFY` $\to$ `OP_EQUALVERIFY`, `OP_NUMEQUAL` `OP_VERIFY` $\to$ `OP_NUMEQUALVERIFY`, `OP_CHECKSIG` `OP_VERIFY` $\to$ `OP_CHECKSIGVERIFY`. Each fusion saves one byte.
2. **Swap elimination**: `OP_SWAP` `OP_SWAP` $\to \varepsilon$. Consecutive swaps cancel out.

Data encoding follows the Bitcoin Script number format:

Definition 4.6 (Script Number Encoding)**.** *A `bigint` value n is encoded as a little-endian, sign-magnitude byte sequence with minimal encoding:*

- $n = 0$*: `OP_0` (opcode `0x00`, pushes empty byte array)*
- $1 \leq n \leq 16$*: `OP_`n (opcodes `0x51`–`0x60`)*
- $n = -1$*: `OP_1NEGATE` (opcode `0x4f`)*

Algorithm 1: BringToTop: Retrieve a value from the stack

Input: Variable name v, boolean *consume*, stack map σ
Output: Sequence of stack operations

$d \leftarrow \sigma^{-1}(v)$; `// Find position of` v `in` σ
if $d = 0$ **then**
  if *consume* **then**
    return ε ; `// Already on top; will be consumed by next op`
  else
    return `OP_DUP` ; `// Copy top item`
  end
else if $d = 1$ ***and*** *consume* **then**
  return `OP_SWAP` ; `// 1-byte swap is cheaper than 2-byte ROLL`
else if $d = 1$ ***and not*** *consume* **then**
  return `OP_OVER` ; `// Copy second item`
else if $d = 2$ ***and*** *consume* **then**
  return `OP_ROT` ; `// Rotate top 3 (equivalent to ROLL 2)`
else
  if *consume* **then**
    return `push`(d); `OP_ROLL`
  else
    return `push`(d); `OP_PICK`
  end
end
Update σ accordingly;

- *Otherwise: little-endian bytes, with sign bit in MSB. If the MSB's high bit is set, an additional `0x00` (positive) or `0x80` (negative) byte is appended.*

Byte string data is pushed using direct push ($\leq$ 75 bytes), `OP_PUSHDATA1` (76–255 bytes), `OP_PUSHDATA2` (256–65535 bytes), or `OP_PUSHDATA4` ($>$ 65535 bytes).

5 Intermediate Representations

The compilation pipeline uses three intermediate representations, each at a different level of abstraction.

5.1 Contract AST

The Contract AST is a tree-structured representation with the following principal node types:

$$
\begin{aligned}
\texttt{ContractNode} &= \langle \textit{name}, \textit{parentClass}, \textit{properties}^*, \textit{constructor}, \textit{methods}^* \rangle \\
\texttt{PropertyNode} &= \langle \textit{name}, \textit{type}, \textit{readonly} \rangle \\
\texttt{MethodNode} &= \langle \textit{name}, \textit{params}^*, \textit{body}^*, \textit{visibility} \rangle \\
\texttt{Statement} &= \texttt{VarDecl} \mid \texttt{Assign} \mid \texttt{If} \mid \texttt{For} \mid \texttt{Return} \mid \texttt{ExprStmt} \\
\texttt{Expression} &= \texttt{BinExpr} \mid \texttt{UnaryExpr} \mid \texttt{CallExpr} \mid \texttt{Ident} \mid \texttt{Literal} \mid \cdots
\end{aligned}
$$

5.2 ANF IR

The ANF IR is a flat, canonically serializable representation. Its structure is:

$$
\begin{aligned}
\texttt{ANFProgram} &= \langle \mathit{version}, \mathit{contractName}, \mathit{properties}^*, \mathit{methods}^* \rangle \\
\texttt{ANFMethod} &= \langle \mathit{name}, \mathit{params}^*, \mathit{body}^*, \mathit{isPublic} \rangle \\
\texttt{ANFBinding} &= \langle t_i, V_i \rangle
\end{aligned}
$$

The value kinds V are:

Kind	Description
`load_param`(x)	Load method parameter x
`load_prop`(x)	Load contract property x
`load_const`(c)	Push compile-time constant c
`bin_op`$(\oplus, l, r)$	Binary operation on temporaries l, r
`unary_op`$(\ominus, x)$	Unary operation on temporary x
`call`$(f, \vec{a})$	Built-in function call
`method_call`$(m, \vec{a})$	Private method call (inlined)
`if`$(c, \vec{t}, \vec{e})$	Conditional with then/else binding sequences
`loop`$(n, \vec{b}, i)$	Unrolled loop of n iterations
`assert`(x)	Runtime assertion
`update_prop`(p, v)	Mutable property update
`get_state_script`()	Serialize state to bytes
`check_preimage`(x)	Verify sighash preimage
`add_output`$(s, \vec{v})$	Emit transaction output

The ANF IR is serialized as JSON with lexicographically sorted object keys and deterministic formatting, producing a canonical form that serves as the *conformance boundary*: all seven compiler implementations must produce byte-identical ANF IR JSON for the same input. The conformance runner canonicalizes by recursively sorting keys and re-serializing with uniform indentation. A strict RFC 8785 (JSON Canonicalization Scheme) serializer is also available in the IR schema package for use cases requiring minimal canonical JSON.

5.3 Stack IR

The Stack IR is a flat sequence of stack machine operations:

$$
\begin{aligned}
\texttt{StackOp} &= \texttt{push}(v) \mid \texttt{dup} \mid \texttt{swap} \mid \texttt{roll}(d) \mid \texttt{pick}(d) \\
&\mid \texttt{drop} \mid \texttt{nip} \mid \texttt{over} \mid \texttt{rot} \mid \texttt{tuck} \\
&\mid \texttt{opcode}(c) \mid \texttt{if}(\vec{t}, \vec{e})
\end{aligned}
$$

where v is a push value (integer, boolean, or byte sequence), d is a stack depth, c is a Bitcoin Script opcode name, and $\vec{t}, \vec{e}$ are sequences of stack operations for the then/else branches.

6 Bitcoin Script Code Generation

This section details how specific language constructs are compiled to Bitcoin Script opcode sequences.

6.1 Operator Compilation

Binary operators map directly to Bitcoin Script opcodes:

Operator	Opcode	Operator	Opcode
+	OP_ADD	=== (int)	OP_NUMEQUAL
-	OP_SUB	=== (bytes)	OP_EQUAL
*	OP_MUL	!== (int)	OP_NUMEQUAL OP_NOT
/	OP_DIV	!== (bytes)	OP_EQUAL OP_NOT
%	OP_MOD	&&	OP_BOOLAND
<	OP_LESSTHAN	||	OP_BOOLOR
>	OP_GREATERTHAN	&	OP_AND
<=	OP_LESSTHANOREQUAL	|	OP_OR
>=	OP_GREATERTHANOREQUAL	^	OP_XOR
«	OP_LSHIFT	»	OP_RSHIFT

Note the type-sensitive equality compilation: the type checker annotates `===` and `!==` operations with a `result_type` field. When comparing byte-family types, `OP_EQUAL` is used; for integer-family types, `OP_NUMEQUAL` is used. This distinction is critical because `OP_NUMEQUAL` interprets stack items as Script numbers with sign-magnitude encoding, while `OP_EQUAL` performs raw byte comparison.

6.2 Built-in Function Compilation

6.2.1 Single-Opcode Built-ins

Many built-in functions compile to a single Bitcoin Script opcode:

Function	Opcode	Function	Opcode
sha256	OP_SHA256	abs	OP_ABS
hash160	OP_HASH160	min	OP_MIN
hash256	OP_HASH256	max	OP_MAX
ripemd160	OP_RIPEMD160	within	OP_WITHIN
checkSig	OP_CHECKSIG	len	OP_SIZE
cat	OP_CAT	bool	OP_0NOTEQUAL

6.2.2 Multi-Opcode Built-ins

More complex built-in functions require multi-opcode sequences. We describe several representative examples.

Power (`pow`). Exponentiation is compiled as a linear conditional multiply with 32 iterations. Given stack state $\langle base, exp \rangle$:

```
OP_SWAP OP_1           // stack: exp base 1(accumulator)
// For i = 0 to 31:
  [2] OP_PICK          // copy exp
  <i+1> OP_GREATERTHAN
  OP_IF
    OP_OVER OP_MUL     // acc *= base
  OP_ENDIF
OP_NIP OP_NIP          // clean up, leave result
```

This computes $base^{\min(exp,32)}$ in exactly 32 conditional multiply steps. The algorithm is identical across all seven compiler implementations for cross-compiler conformance.

Square Root (`sqrt`). Integer square root uses Newton's method with 16 iterations:

```
OP_DUP                  // stack: n guess(=n)
// For i = 0 to 15:
  OP_OVER OP_OVER       // copy n and guess
  OP_DIV OP_ADD         // guess + n/guess
  [2] OP_DIV            // new_guess = (guess + n/guess) / 2
OP_NIP                  // leave sqrt(n)
```

GCD (`gcd`). The Euclidean algorithm with 256 iteration bound:

```
OP_ABS OP_SWAP OP_ABS OP_SWAP   // ensure positive
// For i = 0 to 255:
  OP_DUP OP_0NOTEQUAL
  OP_IF
    OP_TUCK OP_MOD               // a, b -> b, a%b
  OP_ENDIF
OP_DROP                          // leave gcd
```

6.3 Method Dispatch

When a contract has a single public method, the locking script contains that method's body directly. When multiple public methods exist, the compiler generates a dispatch preamble:

$$\underbrace{\texttt{<params>}\ \texttt{<method_index>}}_{\text{unlocking script}}\quad\underbrace{\texttt{OP_DUP}\ 0\ \texttt{OP_NUMEQUAL}\ \texttt{OP_IF}\ \texttt{OP_DROP}\ M_0\ \texttt{OP_ELSE}\ \cdots\ \texttt{OP_ENDIF}}_{\text{locking script}}$$

where each M_i is the compiled body of the i-th public method. The method index is a `bigint` value pushed by the spending transaction.

6.4 Terminal Assert Pattern

Bitcoin Script execution succeeds if and only if the top of the stack is truthy after the script completes. The RÚNAR compiler exploits this by treating the *last* assertion in a public method differently:

- **Non-terminal assert**: compiles to e `OP_VERIFY` (consumes the value, halts on failure)
- **Terminal assert**: compiles to e only (leaves the boolean on the stack for the VM to check)

This optimization saves one byte per public method and is semantically equivalent: if the value is falsy, the VM rejects the transaction.

6.5 Loop Compilation

All loops are *fully unrolled* at compile time. A loop `for (let i = 0; i < n; i++)` produces n copies of the loop body, with the iteration variable i replaced by a constant at each iteration.

This approach is necessitated by Bitcoin Script's lack of backward jumps. The validation pass (Section 4.2) guarantees that n is a compile-time constant, ensuring finite unrolling.

An important subtlety arises with outer-scope variables referenced within loop bodies: in non-final iterations, such variables must be accessed via `OP_PICK` (copying) rather than `OP_ROLL` (consuming), even if the reference appears to be the variable's last use within that iteration. Only in the final iteration may the variable be consumed. The stack lowerer handles this by extending the last-use index for outer-scope variables to the final binding of the final iteration.

7 Stateful Contract Compilation

Stateful contracts implement the *OP_PUSH_TX* pattern, which enables Bitcoin transactions to introspect their own contents and enforce state transitions.

7.1 The OP_PUSH_TX Pattern

The OP_PUSH_TX technique uses `OP_CHECKSIG` with a known private key to verify that a sighash preimage provided in the unlocking script corresponds to the current transaction. The verification proceeds as follows:

1. The spending transaction provides a BIP-143 sighash preimage in its unlocking script.
2. The locking script verifies the preimage using `OP_CHECKSIG` with the well-known public key corresponding to private key $k = 1$ (the compressed generator point G of secp256k1).
3. Once verified, the preimage bytes can be parsed to extract transaction fields: version, outpoint, script code, amount, output hash, locktime, etc.

7.2 State Continuation

After verifying the preimage, the compiler emits code that:

1. Serializes the contract's updated mutable properties into a byte string (the "state script").
2. Computes `hash256` of the serialized state concatenated with the contract's code.
3. Extracts the output hash from the verified preimage.
4. Asserts equality between the computed and extracted hashes.

This ensures that the spending transaction's output contains the contract's code with the updated state, effectively enforcing a state machine transition.

7.3 Multi-Output Contracts

For contracts that produce multiple outputs via `addOutput()`, the compilation emits serialization code for each output:

$$output_i = \mathrm{LE}_{64}(satoshis) \parallel \mathrm{encode}(v_1) \parallel \mathrm{encode}(v_2) \parallel \cdots$$

where LE_{64} is little-endian 64-bit encoding and encode is type-dependent serialization. All outputs are concatenated, hashed, and compared against the preimage's output hash field.

Property values are encoded as follows:

- `bigint`: 8-byte little-endian via `OP_NUM2BIN`
- `boolean`: 1-byte little-endian via `OP_NUM2BIN`
- `ByteString` and subtypes: raw bytes (no transformation)

8 Correctness Properties

The RÚNAR compilation pipeline provides several formal guarantees.

Theorem 8.1 (Type Safety)**.** *If a program passes the type checker (Pass 3), then the generated Bitcoin Script will not encounter a type mismatch at runtime. Specifically, every opcode receives operands of the expected type on the stack.*

Proof sketch. The type checker verifies that every expression's type is consistent with the expected types of the operations that consume it. The ANF lowering preserves these types (each binding carries a type annotation). The stack lowering maintains a typed stack map, and each opcode is emitted only when the stack's type state matches the opcode's signature. □

Theorem 8.2 (Termination)**.** *Every program accepted by the validator (Pass 2) terminates.*

Proof sketch. By Proposition 4.1, all loops are bounded and there is no recursion. Since each builtin operation and opcode terminates, and the program is a finite composition of terminating operations, the entire program terminates. □

Theorem 8.3 (Determinism)**.** *For any valid RÚNAR source program P, the compilation function $\mathcal{C}[\![P]\!]$ is deterministic: the same input always produces the same Bitcoin Script bytecode.*

Proof sketch. Each pass is a pure function with no randomness or external state. The ANF naming scheme ($t_0, t_1, \ldots$) is canonical. The stack lowering algorithm (Algorithm 1) is deterministic given the stack map and last-use analysis, both of which are determined by the ANF IR. □

Theorem 8.4 (Cross-Compiler Conformance)**.** *The seven compiler implementations (TypeScript, Go, Rust, Python, Zig, Ruby, Java) produce byte-identical Bitcoin Script for any valid RÚNAR source program.*

Proof sketch. All seven implementations adhere to the same ANF IR specification (which is canonically serializable) and the same stack lowering algorithm. The ANF IR serves as the conformance boundary: if all seven produce the same ANF IR, and the stack lowering and emission algorithms are specified unambiguously, then the output must be identical. □

Property 8.5 (Stack Safety)**.** *The compiled Bitcoin Script never underflows or overflows the stack. The stack lowerer performs static depth analysis and rejects programs that exceed the maximum stack depth of 800 items.*

Property 8.6 (UTXO Safety)**.** *For stateful contracts, the affine type constraint prevents duplication of `SigHashPreimage` values when passed as identifier arguments to consuming functions, mitigating transaction malleability attacks. Combined with the compiler's automatic injection of `checkPreimage` at public method entry—where the preimage is always an identifier parameter—this ensures that the preimage is consumed exactly once in the standard usage pattern. The state continuation pattern ensures that every state transition produces a valid output matching the expected hash.*

9 Post-Quantum Signature Verification (Experimental)

RÚNAR includes experimental support for post-quantum signature schemes that can be verified entirely within Bitcoin Script, providing resistance against quantum computing attacks on ECDSA. These features are experimental and the APIs may change in future releases.

9.1 WOTS+ (Winternitz One-Time Signature)

The `verifyWOTS` built-in compiles to approximately 10 KB of Bitcoin Script implementing the WOTS+ signature verification algorithm with Winternitz parameter $w = 16$.

The verification proceeds as follows:

1. Hash the message with SHA-256 to produce a 32-byte digest.
2. Process the digest in 4-bit nibbles (64 chains), plus 3 checksum chains (67 chains total).
3. For each chain: extract 15 bytes of the signature, apply up to 15 SHA-256 iterations based on the nibble value, and accumulate the endpoint.
4. Hash the concatenated 67 endpoints to compute the public key.
5. Compare the computed public key against the stored public key.

A critical implementation detail is the *BIN2NUM sign bit pitfall*: single bytes $\geq$ `0x80` are interpreted as negative numbers in Bitcoin Script's number encoding. The compiler appends a `0x00` byte before `OP_BIN2NUM` to ensure correct unsigned interpretation.

The stack lowerer manages the complex stack state using the alt stack for temporary saves during chain processing, maintaining a canonical layout between chains: signature remainder at position 0, checksum at position 1, and endpoint accumulator at position 2.

9.2 SLH-DSA (FIPS 205, SPHINCS+ SHA-256)

SLH-DSA is the NIST-standardized stateless hash-based signature scheme (FIPS 205), based on SPHINCS+. Unlike WOTS+ which is one-time, SLH-DSA supports unlimited signatures from a single keypair, making it suitable for scenarios where key reuse is necessary.

RÚNAR supports all six SHA-256 parameter sets (128s/f, 192s/f, 256s/f) through corresponding built-in functions of the form `verifySLHDSA_SHA2_`*param*.

SLH-DSA verification composes three sub-algorithms:

1. **FORS** (Forest of Random Subsets): A few-time signature scheme that hashes a secret value and verifies a Merkle authentication path for each of k trees of height a. The k tree roots are compressed into a FORS public key.
2. **WOTS+** (Winternitz OTS): Used at each hypertree layer to sign the previous layer's root. Each WOTS+ instance uses *tweakable hashing* with domain separation via a 22-byte compressed address (ADRS).
3. **Hypertree**: A chain of d XMSS trees, each containing a Merkle tree with WOTS+ leaves. Verification walks d layers, each verifying a WOTS+ signature and a Merkle authentication path.

Unlike standalone WOTS+ which uses plain SHA-256, every hash operation in SLH-DSA is a *tweakable hash*:

$$T(\text{PK.seed}, \text{ADRS}, M) = \text{trunc}_n(\text{SHA-256}(\text{PK.seed} \| \text{pad} \| \text{ADRS}_c \| M))$$

where ADRS_c is a 22-byte compressed address providing domain separation between different hash invocations.

Table 3: SLH-DSA SHA-256 parameter sets

Parameter set	n	d	h'	k	a	Sig size	Script size
128s	16	7	9	14	12	7,856 B	203 KB[†]
128f	16	22	3	33	6	17,088 B	∼310 KB
192s	24	7	9	17	14	16,224 B	∼310 KB
192f	24	22	3	33	8	35,664 B	∼410 KB
256s	32	8	8	22	14	29,792 B	∼410 KB
256f	32	17	4	35	9	49,856 B	∼620 KB

[†]Measured from conformance test golden file. Other values estimated by parameter scaling.

The script sizes are substantially larger than WOTS+ (∼10 KB) due to the three-tier tree structure and per-hash domain separation. The measured SHA2-128s script (203 KB) is approximately 5× larger than the earlier estimates, primarily because the tweakable hash operations each require a 64-byte padded seed prefix, inflating push data volume. All parameter sets remain well within Bitcoin SV's 10 MB script size limit.

10 Empirical Evaluation

This section presents empirical measurements of the RÚNAR compiler across its test corpus, covering cross-compiler conformance, script sizes, and test suite structure.

10.1 Test Corpus

The RÚNAR test suite comprises three tiers of validation:

1. **Per-pass unit tests** (11 test files): Each compiler pass has dedicated tests covering parsing, validation, type checking, ANF lowering, stack lowering, emission, optimization, and multi-format dispatch.
2. **Contract-level tests** (∼50 contract implementations): 12 base contracts implemented across up to 5 source formats, each with business-logic tests exercising constructor initialization, method calls, state transitions, and assertion behavior.
3. **Cross-compiler conformance tests** (9 golden-file tests + dynamic discovery): The conformance suite compiles each test case through all seven compiler implementations and compares the output against a golden `expected-script.hex` file, requiring byte-identical results.

Table 4 summarizes the test corpus by source format.

Source Format	Contracts	Compilers	Status
TypeScript (`.runar.ts`)	12	TS, Go, Rust	Stable
Solidity-like (`.runar.sol`)	9	TS, Go, Rust	Experimental
Move-style (`.runar.move`)	9	TS, Go, Rust	Experimental
Go (`.runar.go`)	10	Go	Experimental
Rust (`.runar.rs`)	10	Rust	Experimental
Total	**50**		

Table 4: Test corpus: contract implementations by source format.

10.2 Cross-Compiler Conformance Results

The conformance suite tests 9 golden-file cases and dynamically discovers all example contracts for cross-compilation. Table 5 lists the golden-file test cases with their output sizes.

Test Case	Script Size (bytes)	Category
basic-p2pkh	5	Signature verification
if-else	18	Conditional branching
multi-method	24	Method dispatch
boolean-logic	31	Boolean operators
arithmetic	35	Integer arithmetic
bounded-loop	51	Loop unrolling
stateful	129	OP_PUSH_TX + state continuation
post-quantum-wots	10,530	WOTS+ hash-based signature
post-quantum-slhdsa	207,874	SLH-DSA (SHA2-128s)

Table 5: Conformance test cases with byte-identical output across all seven compiler implementations (TypeScript, Go, Rust, Python, Zig, Ruby, Java).

All 9 conformance tests produce byte-identical Bitcoin Script across the TypeScript, Go, and Rust compilers. The cross-compiler test suite additionally compiles all 12 TypeScript example contracts through both the TS and Go pipelines, verifying identical hex output.

10.3 Script Size Measurements

Table 6 reports compiled script sizes for the full example contract suite.

Several observations emerge from these measurements:

Contract	Size (B)	Opcodes	Notes
P2PKH	5	5	Stateless, 1 method
Escrow	30	30	Stateless, multi-sig
OraclePriceFeed	41	38	Rabin oracle
CovenantVault	42	9	Covenant pattern
NFTExample	112	44	Stateful, multi-output
Counter	130	60	Stateful, state continuation
Auction	142	74	Stateful, multi-method
FungibleTokenExample	259	154	Stateful, multi-output
FunctionPatterns	373	231	Private method inlining
MathDemo	2,548	2,215	Extended math builtins
PostQuantumWallet	10,530	10,461	WOTS+ (67 chains)

Table 6: Compiled script sizes for the example contract suite. All contracts produce byte-identical output across the TS, Go, and Rust compilers.

- **Typical contracts are compact.** The eight contracts representing common smart contract patterns (P2PKH through FungibleTokenExample) range from 5 to 259 bytes, well within Bitcoin SV's standard transaction limits.
- **Loop unrolling dominates size.** `MathDemo` exercises math builtins that expand into multi-opcode sequences (`sqrt`: 256 Newton rounds, `pow`: 32 conditional multiplies, `gcd`: 256 Euclidean iterations), resulting in 14.8 KB of script. This illustrates the size–generality tradeoff inherent in full loop unrolling.
- **Post-quantum scripts are large but feasible.** `PostQuantumWallet` at 10.5 KB is 2–3 orders of magnitude larger than typical contracts, but remains well under Bitcoin SV's 10 MB script size limit.
- **Opcode-to-byte ratio varies.** Contracts with many push data operations (CovenantVault, NFTExample) have lower opcode-to-byte ratios due to inline data, while computation-heavy contracts (MathDemo, PostQuantumWallet) have ratios near 1:1.

10.4 Test Infrastructure

The CI pipeline validates all seven compilers independently:

1. **TypeScript**: `pnpm test` runs all package-level unit tests and example contract tests via vitest.
2. **Go**: `go test ./...` with race detection runs compiler tests and native Go contract tests.
3. **Rust**: `cargo test` runs compiler tests and native Rust contract tests.
4. **Conformance**: A dedicated job (dependent on all seven) compiles each conformance test through every compiler, comparing hex output against golden files.

In addition to compilation testing, the BSV SDK script execution tests (`ScriptExecutionContract`) compile contracts with baked constructor arguments and execute them through the `@bsv/sdk` `Spend` class, validating that the generated Bitcoin Script executes correctly in a production-grade script interpreter.

10.5 Deployment SDKs

Deployment SDKs are provided in all seven implementation languages (TypeScript, Go, Rust, Python, Zig, Ruby, Java), enabling contract deployment, method invocation, state management, and blockchain interaction from any language. Each SDK provides equivalent functionality: contract lifecycle management (`RunarContract`), blockchain abstraction (`Provider` interface with mock and WhatsOnChain implementations), and signing abstraction (`Signer` interface with local and external implementations). The TypeScript SDK delegates cryptographic operations

to the official BSV TypeScript SDK (`@bsv/sdk`) via its `LocalSigner`. The Go and Rust SDKs provide an `ExternalSigner` callback pattern that lets callers integrate any signing library (e.g., `go-sdk` for Go, `rust-sv` for Rust) without introducing direct crypto dependencies into the SDK packages.

11 Limitations and Threats to Validity

We identify the following limitations of the current work.

Proof sketches, not mechanized proofs. The correctness properties in Section 8 are stated as theorems with proof sketches, not as machine-checked proofs in a proof assistant (e.g., Coq or Lean). In particular, the type safety theorem (Theorem 8.1) relies on an informal argument about type-state propagation through the stack lowerer. A mechanized formalization would strengthen these claims but is beyond the current scope.

Conformance is empirical, not proven. Cross-compiler conformance (Theorem 8.4) is validated by a test suite of 9 golden-file cases plus dynamic discovery over 12 example contracts. While no discrepancies have been observed, the test corpus does not exhaustively cover all language features or their combinations. Corner cases involving complex interactions between loop unrolling, conditional branches, and property updates may exist that are not covered by the current suite.

Limited affine tracking. The affine type discipline applies only to two types (`SigHashPreimage` and `Sig`). This is a targeted constraint rather than a general linear or affine type system. The type checker uses a single shared consumption set across conditional branches: if an affine value is consumed in the then-branch, the else-branch will reject it as already consumed. This is overly conservative compared to a split-environment affine system (which would permit consumption in both branches, since exactly one executes at runtime), but remains sound—it rejects some valid programs rather than accepting invalid ones. Additionally, affine tracking only applies when values are passed as identifier arguments to consuming functions, not to arbitrary expressions.

Loop unrolling size overhead. Full loop unrolling is the only compilation strategy for bounded loops, producing $O(n)$ copies of the loop body. For contracts with large iteration counts or deeply nested loops, this can result in script sizes that are orders of magnitude larger than the source code. Partial unrolling or runtime loop constructs (which would require Bitcoin Script extensions) could mitigate this, but are not currently supported.

Post-quantum features are experimental. Both WOTS+ and SLH-DSA script generation are implemented across all seven compilers (TypeScript, Go, Rust, Python, Zig, Ruby, Java) and produce byte-identical output in conformance testing. However, the resulting scripts are large: SLH-DSA SHA2-128s generates approximately 203 KB of Bitcoin Script (Table 5), and larger parameter sets produce proportionally larger scripts. While these sizes are within Bitcoin SV's 10 MB limit, the practical deployment cost of such large scripts remains to be evaluated. The post-quantum APIs are experimental and may change in future releases.

Go and Rust frontend parity. The Go frontend (`.runar.go`) and the Rust frontend (`.runar.rs`) were originally single-compiler formats, but all seven compilers now ship parsers for both, so cross-compiler conformance holds for these formats as well.

No formal Bitcoin Script semantics. The correctness arguments assume but do not formalize the semantics of Bitcoin Script itself. The compiler targets the BSV node's interpreter behavior as the de facto specification. Discrepancies between node implementations, or future opcode semantics changes, could invalidate the compiled output without changes to the RÚNAR compiler.

12 Related Work

Several projects have addressed the challenge of compiling high-level languages to Bitcoin Script or similar stack-based targets.

sCrypt [1] provides a TypeScript-based DSL for Bitcoin SV smart contracts, sharing a similar source language design with RÚNAR. However, sCrypt uses a single compiler implementation, whereas RÚNAR employs seven independent implementations with a formal conformance boundary.

Miniscript [2] is a structured subset of Bitcoin Script focused on composability and analysis of spending policies. Unlike RÚNAR, Miniscript does not support general computation or state management, instead focusing on policy composition from a fixed set of primitives.

Ivy [3] is a high-level language for Bitcoin that compiles to Script. Ivy uses a simpler type system and does not support stateful contracts or multiple frontend syntaxes.

CashScript [4] targets Bitcoin Cash and compiles a Solidity-like language to Bitcoin Script. It shares the Solidity-inspired surface syntax approach with one of RÚNAR's frontends but supports only a single source language.

Move [5] is a resource-oriented language for the Diem/Aptos blockchain with a linear type system. RÚNAR's Move-style frontend draws syntactic inspiration from Move but targets a fundamentally different execution model (stack-based UTXO vs. account-based).

The multi-frontend approach of RÚNAR is inspired by compiler frameworks such as GCC and LLVM, which separate frontend parsing from backend code generation through a shared intermediate representation.

13 Conclusion

We have presented RÚNAR, a multi-frontend compiler that translates smart contracts from five high-level languages into Bitcoin SV Script through a six-phase pipeline. The key contributions are:

(i) A **multi-frontend architecture** that demonstrates language-agnostic smart contract compilation, allowing developers to write Bitcoin contracts in their preferred syntax.

(ii) A **canonically serializable ANF IR** that serves as a formal conformance boundary between seven independent compiler implementations, enabling cross-compiler verification of correctness.

(iii) A **deterministic stack scheduling algorithm** that maps named temporaries to positional stack operations with last-use analysis, producing optimal stack manipulation sequences.

(iv) **Static guarantees** of type safety, termination, determinism, and UTXO safety through the combination of a restricted type system, bounded iteration, targeted affine consumption tracking, and static stack analysis.

(v) **Post-quantum signature verification** in Bitcoin Script via WOTS+, demonstrating that non-trivial cryptographic protocols can be compiled from high-level specifications.

The RÚNAR approach shows that the gap between high-level smart contract languages and low-level stack-based execution can be bridged systematically, with formal guarantees preserved across the compilation pipeline. The multi-frontend design validates that Bitcoin Script's expressiveness is independent of surface syntax: the same computational patterns can be expressed naturally in TypeScript, Solidity, Move, Go, and Rust idioms, all converging onto identical Bitcoin Script bytecode.

As discussed in Section 11, several avenues remain for future work: mechanizing the correctness proofs, expanding the conformance test corpus to achieve stronger coverage guarantees, and extending cross-compiler conformance to the Go and Rust frontend formats.

References

[1] Liu, X. et al. *sCrypt: A High-Level Smart Contract Language for Bitcoin SV*. sCrypt Inc., 2020. `https://scrypt.io`

[2] Wuille, P., Poelstra, A., and Sanchez, E. *Miniscript: A Structured Representation of Bitcoin Script*. 2019. `https://bitcoin.sipa.be/miniscript/`

[3] Chain Inc. *Ivy: A Smart Contract Language for Bitcoin*. 2017. `https://ivy-lang.org`

[4] De Meester, R. *CashScript: A High-Level Language for Bitcoin Cash*. 2019. `https://cashscript.org`

[5] Blackshear, S., Cheng, E., Dill, D. L., Gao, V., Maurer, B., and Nowacki, T. *Move: A Language With Programmable Resources*. Diem Association, 2019.

[6] Lau, J. and Wuille, P. *BIP 143: Transaction Signature Verification for Version 0 Witness Program*. Bitcoin Improvement Proposals, 2016.

[7] Flanagan, C., Sabry, A., Duba, B. F., and Felleisen, M. *The Essence of Compiling with Continuations*. ACM SIGPLAN Notices, 28(6):237–247, 1993.

[8] Hülsing, A. *W-OTS+ – Shorter Signatures for Hash-Based Signature Schemes*. In *AFRICACRYPT 2013*, LNCS 7918, pages 173–188. Springer, 2013.